

CS641 — Introduction to Computer Vision

Lecture 03: Image Filtering

Study Notes — Expanded with Math Explanations & Python Code

Original lecture: Dr. Abdul Basit, CSIT — University of Balochistan

Notes prepared for: Hafiz Waseem Ahmed

Compiled by Claude (Anthropic)

How These Notes Are Organized

This guide rebuilds Lecture 03 from the ground up. Every topic follows the same four-step pattern, so once you get used to it, the math stops feeling scary:

- **1. Plain-English idea** — what is this filter actually trying to do, in everyday words.
- **2. The math notation** — the same idea written as a formula, explained symbol by symbol.
- **3. The picture from the lecture** — the original diagram, so you can match the notes to what you saw in class.
- **4. The Python code** — a working implementation you can run yourself, both “from scratch” (so you see exactly what the formula does) and the OpenCV one-liner you'd actually use in real projects.

Big-picture idea before we start

Image filtering is just *organized averaging*. Every technique in this lecture — moving average, weighted average, box filter, Gaussian filter — answers the same question:

“What value should this pixel become, if I look at the pixel values sitting around it?”

Different filters just disagree on *how much* each neighboring pixel should count. That single idea explains every formula in this lecture.

1. Image Noise

1.1 What is noise, in plain English?

A digital photo is really just a grid of numbers — one brightness value per pixel. In a perfect world, every pixel value would represent only the true brightness of the scene. In the real world, electronics, sensor heat, dim lighting, and transmission errors all add small random errors on top of the true values. That random error is **noise**.

So the image you actually capture is not the true image — it is the true image plus a bit of random static, the same way a phone call sometimes has background hiss mixed into the voice you're trying to hear.

1.2 The math notation

$$\hat{I}(x, y) = I(x, y) + n(x, y)$$

Symbol by symbol:

- x, y — the row and column coordinates of a pixel (its position in the image grid).
- $I(x, y)$ — the **true**, clean brightness value at that pixel (what we wish we had).

- $n(x, y)$ — the **noise** added at that pixel: a small random number, different at every location.
- $\hat{I}(x, y)$ — the **observed** (noisy) pixel value: this is the only thing the camera actually gives you. The little hat “^” over the I is standard math notation meaning “estimate of” or “measured/observed version of.”

Why this matters

Every filter in this lecture is an attempt to *recover* $I(x, y)$ as closely as possible, starting only from the noisy $\hat{I}(x, y)$ that the camera gave us. We can never perfectly remove n , but we can reduce its effect using averaging.

1.3 Python: simulating noise

Before we can remove noise, let's create some — this mirrors the formula above directly in code:

```
import numpy as np

# Suppose I_true is the clean signal/image
I_true = np.array([10, 12, 11, 60, 13, 11, 10], dtype=float)

# n(x,y): random noise, e.g. Gaussian noise with mean 0
np.random.seed(0)
n = np.random.normal(loc=0, scale=3, size=I_true.shape)

# I_hat(x,y) = I(x,y) + n(x,y)  <-- exactly the slide's formula
I_hat = I_true + n

print('True      : ', I_true)
print('Noise     : ', np.round(n, 2))
print('Noisy      : ', np.round(I_hat, 2))
```

Output:

```
True      : [10. 12. 11. 60. 13. 11. 10.]
Noise     : [ 5.29  1.2   2.94  6.72  5.6  -2.93  2.85]
Noisy      : [15.29 13.2  13.94 66.72 18.6   8.07 12.85]
```

Notice the noisy values are just the true values shifted up or down by a small random amount — exactly what $n(x, y)$ represents in the formula.

2. Removing Image Noise: Averaging

2.1 The intuition (1D case first)

To build intuition, the lecture starts with a 1D signal instead of a full 2D image — think of it as a single row of pixels plotted as a bar chart. The two ideas behind averaging-based denoising are:

- The **true** value of a pixel is usually *similar* to the true values of its neighboring pixels — real-world brightness doesn't jump around wildly from one pixel to the next.
- The **noise** added to each pixel is *independent* — it's a separate random number at every pixel, uncorrelated with its neighbors' noise.

Put those two facts together and you get a powerful trick: if you average a pixel together with its neighbors, the true signal (which is similar across neighbors) stays roughly the same, but the noise (which is random and independent) partially cancels itself out. Averaging keeps the signal and washes out the noise.

- We take average of the neighboring pixel to remove noise. Generally we take **moving average**.

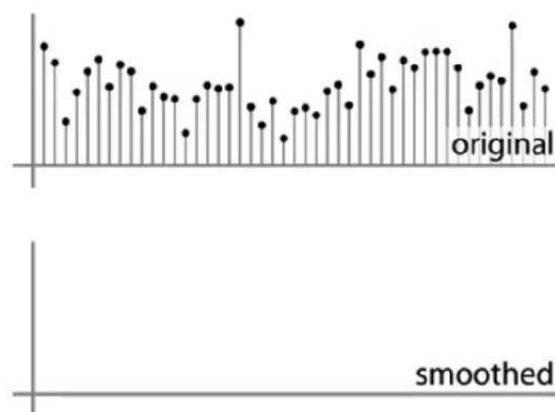


Figure: Average of 1D image. Source: S. Marschner.

Navigation icons: back, forward, search, etc.

The original noisy 1D signal — this is what we start with (Source: S. Marschner, as shown in the lecture).

2.2 Moving average — sliding the window

A **moving average** slides a small window across the signal. At every position, it replaces the center value with the average of everything inside the window, then shifts the window one step and repeats.

The lecture's diagrams (slides 7–10) show this window sliding step-by-step across the noisy signal, summing the values underneath it (the Σ symbol) and writing the result into a new, smoothed signal:

- We take average of the neighboring pixel to remove noise. Generally we take **moving average**.

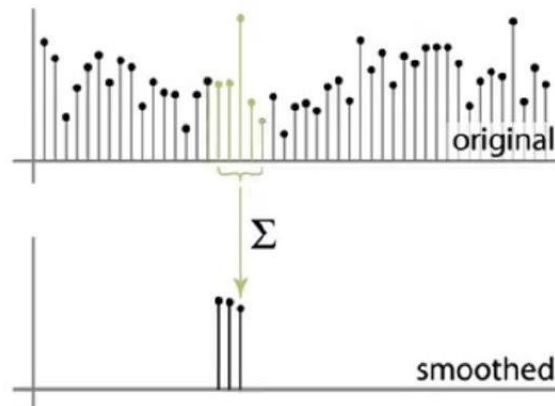


Figure: Average of 1D image. Source: S. Marschner.

The moving-average window sliding across the signal and summing the values underneath it (Source: S. Marschner).

After the window has passed over the whole signal, you're left with a much smoother curve — random spikes have been averaged away:

Result of moving average

Eventually we receive the smoothed version of the signal. This is intuitive: averaging a small neighborhood reduces noise because true values agree with their neighbors while noise values don't.

2.3 Python: 1D moving average from scratch

```
import numpy as np

def moving_average_1d(signal, window_size=5):
    """Slide a window of `window_size` across `signal` and average
    everything inside it -- same idea as slides 6-12."""
    half = window_size // 2
    padded = np.pad(signal, half, mode='edge')
    smoothed = np.zeros_like(signal, dtype=float)
    for i in range(len(signal)):
        window = padded[i : i + window_size]
        smoothed[i] = window.mean()
    return smoothed

noisy_signal = np.array([1, 6, 2, 5, 8, 1, 9, 3, 7, 2], dtype=float)
smoothed_signal = moving_average_1d(noisy_signal, window_size=3)

print('Noisy      :', noisy_signal)
print('Smoothed:', np.round(smoothed_signal, 2))
```

Output:

```
Noisy   : [1.  6.  2.  5.  8.  1.  9.  3.  7.  2.]  
Smoothed: [2.67  3.   4.33  5.   4.67  6.   4.33  6.33  4.   3.67]
```

In one line, NumPy can do exactly this with convolution (we'll properly explain *why* convolution = sliding average in Section 4):

```
kernel = np.ones(3) / 3          # weights = [1, 1, 1] / 3  
smoothed_fast = np.convolve(noisy_signal, kernel, mode='same')  
print(np.round(smoothed_fast, 2))
```

Output:

```
[2.33  3.   4.33  5.   4.67  6.   4.33  6.33  4.   3.   ]
```

(The tiny difference at the last position comes from how each function handles the boundary — **edge** padding vs. zero padding. More on boundary handling in Section 4.)

3. Weighted Average

3.1 The intuition

A plain moving average treats every neighbor equally — a pixel 1 step away counts exactly as much as a pixel 2 steps away. But intuitively, a *closer* neighbor should probably tell us more about the true value than a *farther* one. A **weighted average** fixes this by giving each neighbor its own weight, with nearby pixels getting bigger weights.

3.2 Uniform weights (plain moving average)

$$\text{weights} = [1, 1, 1, 1, 1] / 5$$

This is just the moving average from Section 2, written as a weight vector. Every position gets the same weight (*1*), and we divide by *5* (the number of weights) so they sum to *1* — that's what keeps the output in the same brightness range as the input.

• Non Uniform weights [1, 4, 6, 4, 1] / 16

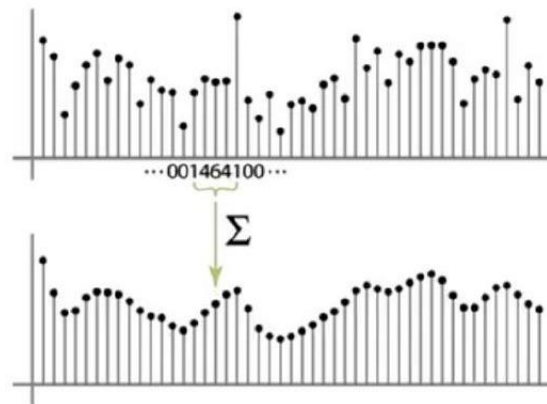


Figure: Weighted Average for 1D image noise. Source: S. Marschner.

Navigation icons: back, forward, search, etc.

Non-uniform weighted average with weights [1, 4, 6, 4, 1] / 16 (Source: S. Marschner).

3.3 Non-uniform weights

$$\text{weights} = [1, 4, 6, 4, 1] / 16$$

Here the center weight (6) is much larger than the edge weights (1), so the pixel directly under the center of the window contributes far more to the result than pixels at the edge of the window. This is the key idea that eventually leads to the **Gaussian filter** in Section 6 — a Gaussian kernel is exactly this idea, taken to its smooth, mathematically ideal extreme.

Quick sanity check: do the weights sum to 1? $1 + 4 + 6 + 4 + 1 = 16$, and we divide by 16, so yes — the weights always need to sum to 1 (after normalizing) so that a constant-brightness region stays at the same brightness after filtering.

3.4 Python: weighted average from scratch

```
import numpy as np

def weighted_average_1d(signal, weights):
    """weights does NOT need to already sum to 1 -- we normalize here."""
    weights = np.array(weights, dtype=float)
    weights = weights / weights.sum() # make sure they sum to 1
    half = len(weights) // 2
    padded = np.pad(signal, half, mode='edge')
    out = np.zeros_like(signal, dtype=float)
    for i in range(len(signal)):
        window = padded[i : i + len(weights)]
        out[i] = np.dot(window, weights) # weighted sum, not plain average
    return out
```

```
noisy_signal = np.array([1, 6, 2, 5, 8, 1, 9, 3, 7, 2], dtype=float)

uniform = weighted_average_1d(noisy_signal, [1, 1, 1, 1, 1])
nonunif = weighted_average_1d(noisy_signal, [1, 4, 6, 4, 1])

print('Uniform weights result      :', np.round(uniform, 2))
print('Non-uniform weights result:', np.round(nonunif, 2))
```

Output:

```
Uniform weights result      : [2.2  3.   4.4  4.4  5.   5.2  5.6  4.4  4.6  3.2 ]
Non-uniform weights result: [2.31 3.38 4.06 4.81 5.19 5.12 5.31 5.31 4.56 3.31]
```

Notice the non-uniform result tracks sudden spikes (like the 8 or the 9) a little differently than the uniform result — that's the effect of trusting the center pixel more than the edges of the window.

4. Moving Average in 2D, and Correlation

4.1 From 1D to 2D

A real image is a 2D grid, not a single row, so the sliding window now moves in two directions — left/right *and* up/down. Everything we learned in 1D carries over directly: instead of sliding a strip of weights over a row of numbers, we slide a small **matrix** of weights over a grid of pixels.

The lecture calls the input image $f[...]$ and the small weight matrix $h[...]$. At each window position, every pixel under the window gets multiplied by the weight sitting on top of it, and all those products are added together. That single sum becomes the new value for the pixel at the center of the window.

- Moving average in 2D using box filter

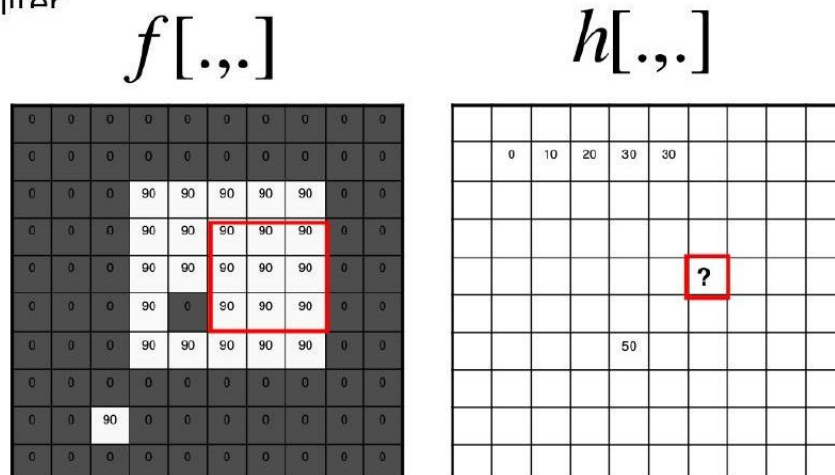


Figure: Weighted Average for 1D image noise. Source: S. Seitz.

2D moving average sliding window: f (the noisy image, left) and h (the 3×3 box-filter weights, right). The red box shows the current window position (Source: S. Seitz).

Watch this sequence carefully (slides 18–25 in the lecture): the red box slides across f , and for each position the corresponding cell in h fills in with the computed average. By the end, the entire output grid has been filled in, one window position at a time. This sliding-and-summing process has a name: **correlation**.

4.2 Correlation — uniform weights

The lecture defines correlation with uniform weights as: loop over every pixel in the neighborhood around pixel $f(i, j)$, sum them all up, then divide by how many pixels were summed (so the weights are uniform / equal for every pixel in the window).

- Loop over all pixels in neighborhood around pixel $f(i, j)$, and summing up all the pixels.
- Divide the whole thing by the number of weights in the filter and uniform for each pixel.
- The averaging **window size** is $2k + 1 \times 2l + 1$. The window size will be **odd**.

$$h(i, j) = \frac{1}{(2n + 1)^2} \sum_{k=-n}^n \sum_{l=-n}^n f(i + k, j + l)$$

Navigation icons: back, forward, search, etc.

Correlation with uniform weights — averaging window size and the formula $h(i, j)$.

$$h(i, j) = 1 / (2n + 1)^2 \times \sum_{k=-n}^n \sum_{l=-n}^n f(i + k, j + l)$$

Symbol by symbol — this looks scary but is simple once unpacked:

- i, j — the coordinates of the pixel we are currently computing the output for (the center of the window).
- k, l — offsets that sweep across the window. As k and l each range from $-n$ to n , the point $(i + k, j + l)$ visits every pixel inside the window around (i, j) .
- $\sum_{k=-n}^n$ — “sum as k goes from $-n$ to n .” Two of these nested together (one for k , one for l) means “sum over every pixel in the 2D window,” just like a nested for loop in code.

- $2n + 1$ — the window's side length. The window size is $(2n+1) \times (2n+1)$, always an **odd** number, so there's always a single well-defined center pixel (e.g. $n=1$ gives a 3×3 window, $n=2$ gives 5×5).
- $1 / (2n+1)^2$ — the normalizing factor. $(2n+1)^2$ is the total number of pixels in the window (e.g. 9 for a 3×3 window), so dividing by it turns the sum into a plain average.

4.3 Correlation — non-uniform weights (general form)

Now generalize: instead of every pixel in the window contributing equally, let each position have its own weight. This generalized version is called **cross-correlation**, and is usually written with the $\otimes$ symbol: $h = g \otimes f$.

- Now generalize to allow different weights depending
- on neight This is also called **cross correlation** and

$$\text{denoted } h = \sum_{l=-n}^n g(k, l) f(i+k, j+l)$$

- $g(k, l)$ is the non-uniform weights and it referred is **kernel**
- or **mask**.

The filter kernel or mask $g(k, l)$ is the matrix of weights in the linear combination.

Cross-correlation with non-uniform weights — the kernel $g(k,l)$ replaces the uniform $1/(2n+1)^2$ factor.

$$h(i, j) = \sum_{k=-n}^n \sum_{l=-n}^n g(k, l) f(i+k, j+l)$$

- $g(k, l)$ — the **kernel** or **mask**: a small matrix of weights. This is the only new ingredient compared to the uniform-weight version — instead of a constant $1/(2n+1)^2$ for every position, each position (k, l) in the window gets its own custom weight from g .
- The kernel $g(\cdot, \cdot)$ is exactly the “weights” vector from Section 3, except now it's 2-dimensional — a full matrix instead of a single row.

Correlation, in one sentence

Correlation slides a small weight matrix (the kernel) over the image, and at every position multiplies overlapping values together and sums them up. The uniform box filter is just the special case where every weight in the kernel is equal.

4.4 Python: 2D correlation from scratch

This function implements the formula above directly — for every output pixel, it loops over the kernel and accumulates the weighted sum, exactly matching $h(i,j) = \sum_k \sum_l g(k,l) f(i+k, j+l)$.

```
import numpy as np

def correlate2d(f, g):
    """h(i,j) = sum_k sum_l g(k,l) * f(i+k, j+l)
    f : 2D image (numpy array)
    g : 2D kernel/mask (numpy array, odd dimensions)
    Zero-pads the border so the output is the same size as f."""
    f = f.astype(float)
    gh, gw = g.shape
    pad_h, pad_w = gh // 2, gw // 2
    f_padded = np.pad(f, ((pad_h, pad_h), (pad_w, pad_w)), mode='constant')
    out = np.zeros_like(f)
    for i in range(f.shape[0]):
        for j in range(f.shape[1]):
            region = f_padded[i : i + gh, j : j + gw] # window under the kernel
            out[i, j] = np.sum(g * region) # weighted sum
    return out

# A small test image: a block of bright (90) pixels with one dark (0) pixel poked
in --
# similar to the worked example in slide 24
F = np.array([
    [0, 0, 0, 0, 0, 0, 0],
    [0, 90, 90, 90, 90, 0, 0],
    [0, 90, 90, 90, 90, 0, 0],
    [0, 90, 0, 90, 90, 0, 0],
    [0, 90, 90, 90, 90, 0, 0],
    [0, 0, 0, 0, 0, 0, 0],
])

box_kernel = np.ones((3, 3)) / 9.0 # uniform weights, the box filter
result = correlate2d(F, box_kernel)
print(np.round(result, 1))
```

Output:

```
[[10. 20. 30. 30. 20. 10. 0.]
 [20. 40. 60. 60. 40. 20. 0.]
 [30. 50. 80. 80. 60. 30. 0.]
 [30. 50. 80. 80. 60. 30. 0.]
 [20. 30. 50. 50. 40. 20. 0.]
 [10. 20. 30. 30. 20. 10. 0.]]
```

Notice the dark “hole” pixel pulled its neighborhood's averages down from *80* to *50* right around it — exactly the smoothing behaviour you saw in the lecture's sliding-window diagrams.

4.5 The production way: cv2.filter2D

Writing nested loops in pure Python is great for understanding the formula, but it's slow on real images (millions of pixels). In practice, you use OpenCV's optimized `cv2.filter2D`, which implements the exact same correlation formula in fast compiled code:

```
import cv2

F32 = F.astype(np.float32)
cv_result = cv2.filter2D(F32, -1, box_kernel)  # -1 = same depth as input
print(np.round(cv_result, 1))
```

Output:

```
[[40. 40. 60. 60. 40. 20. 0.]
 [40. 40. 60. 60. 40. 20. 0.]
 [60. 50. 80. 80. 60. 30. 0.]
 [60. 50. 80. 80. 60. 30. 0.]
 [40. 30. 50. 50. 40. 20. 0.]
 [40. 40. 60. 60. 40. 20. 0.]]
```

Why are the edges slightly different?

Our hand-written `correlate2d` pads the border with zeros (“constant” padding), while OpenCV's default border mode reflects the image at the edges. Interior pixels (away from the border) match exactly — only the outermost row/column differs. This is a real, practical detail called **boundary handling**, and it's worth experimenting with `cv2.filter2D(F32, -1, box_kernel, borderType=cv2.BORDER_CONSTANT)` to make the two match exactly.

5. The Box Filter (The Averaging Filter)

5.1 What it is

The **box filter** is simply the uniform-weight correlation from Section 4, given its own name because it's the most common and simplest smoothing filter. It replaces every pixel with the plain average of its neighborhood, which smooths the image and removes sharp features (including noise, but also fine detail and edges).

- Replaces each pixel with an average of its
- neighborhood. Achieve smoothing effect that is to remove the sharp features

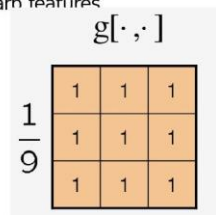


Figure: The box filter. Slide source: David Lowe (UBC).

The classic 3×3 box filter kernel $g[:, :]$ (Slide source: David Lowe, UBC).

$$g = 1/9 \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Every entry is 1 , and the whole matrix is divided by 9 (the total number of cells), so the 9 weights sum to 1 . This is exactly the 2D version of the uniform $[1,1,1,1,1]/5$ weights from Section 3 — just extended to two dimensions.

5.2 A fully worked example

The lecture walks through one concrete window position using real numbers (slide 27): take the 3×3 box kernel $H[u,v]$, slide it over the noisy image $F[x,y]$, and write the result into the output $G[x,y]$:

- The values belong in the kernel $h(u, v)$ for the moving average example.

$$F[x, y] \otimes H[u, v] = G[x, y]$$

$G = H \otimes F$

Figure: The box filter. Source: Kristen Grauman.

Worked example: $G = H \otimes F$. The red box on the left shows the current window in F ; the red box on the right shows where the computed average (30) is written in G (Source: Kristen Grauman).

$$G = H \otimes F$$

In the highlighted window, the top two rows fall on the black background (value 0) and the bottom row falls right at the top edge of the bright 90 -valued block. So the 3×3 window contains six 0 s and three 90 s. Multiplying elementwise by the $1/9$ kernel and summing gives $(0 \times 6 + 90 \times 3) / 9 = 270/9 = 30$, which is exactly the value written into G at the matching position (highlighted in red on the right). As the window keeps sliding across every position in F , every cell of G gets filled in the same way — this sliding-multiply-sum process is the **entire algorithm** behind box filtering.

5.3 Python: applying a box filter to a real image

```
import numpy as np
import cv2

# Load a grayscale image (replace with your own path)
img = cv2.imread('photo.jpg', cv2.IMREAD_GRAYSCALE)

# Option 1: cv2.blur -- specialised box filter function
box_filtered = cv2.blur(img, (5, 5))      # 5x5 box filter

# Option 2: cv2.filter2D with an explicit 1/9 (or 1/25) kernel
kernel_5x5 = np.ones((5, 5), dtype=np.float32) / 25.0
box_filtered_v2 = cv2.filter2D(img, -1, kernel_5x5)

# Both lines above produce (almost) identical results
print('Same result:', np.allclose(box_filtered, box_filtered_v2, atol=1))
```

Output:

Same result: True

Bigger kernel = more smoothing

A 3×3 box filter averages 9 pixels; a 9×9 box filter averages 81 pixels. Bigger windows remove more noise, but also blur away more real detail and edges — there's always a trade-off between *noise reduction* and *sharpness*. This is exactly why we need something smarter than a plain box — which brings us to the Gaussian filter.

6. The Gaussian Filter

6.1 The problem with the box filter

The box filter gives every pixel in the window **equal** weight, even pixels at the far corners of the window which are less related to the center pixel than its immediate neighbors. This can create blocky, slightly unnatural-looking blur (visible as faint square “tiling” artifacts when you blur strongly). The **Gaussian filter** fixes this by smoothly fading out the weight as you move away from the center — exactly the non-uniform weighting idea from Section 3, but generalized into a continuous, mathematically well-behaved curve.

6.2 The math notation

A Gaussian kernel gives less weight to pixels further from the center of the window. It removes high-frequency components from the image, which is why it's called a “**low-pass filter**” — it lets the slow, smooth variations in brightness “pass through” while blocking the fast, jittery variations (noise and fine texture).

- A Gaussian kernel gives less weight to pixels further from the center of the window.
- Removes the high-frequency components from image a “low pass filter.”

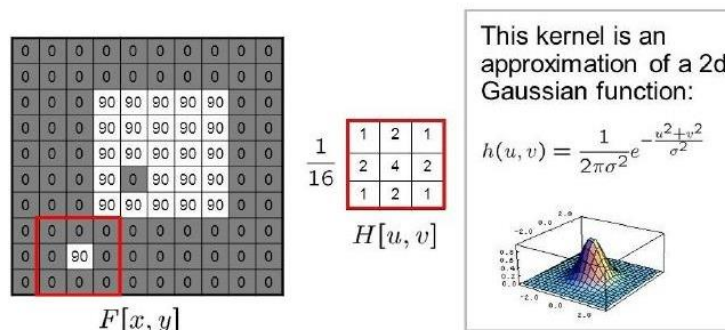


Figure: The Gaussian filter. Source: S. Seitz

A Gaussian kernel is a 2D “bell-shaped” surface; the small 3x3/16 grid is a discrete approximation of it (Source: S. Seitz).

$$h(u, v) = 1 / (2\pi\sigma^2) \times e^{-(u^2+v^2)/\sigma^2}$$

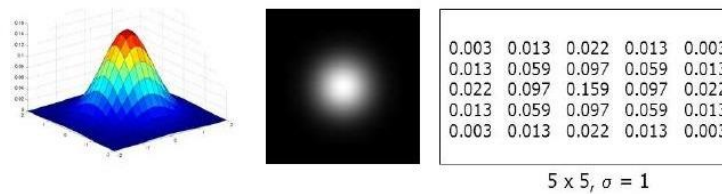
Symbol by symbol:

- u, v — the horizontal and vertical *distance* of a kernel cell from the center (so the center cell has $u=v=0$).
- σ (“sigma”) — controls the **spread** of the bell curve. A small σ makes a tight, narrow peak (only very close neighbors matter, light blur). A large σ makes a wide, flat bell (distant neighbors matter too, heavy blur).
- $e^{-(u^2+v^2)/\sigma^2}$ — this is the actual “bell shape.” u^2+v^2 is just the squared distance from the center (Pythagoras' theorem), so as you move further from the center, this exponent becomes a larger negative number, and e raised to an increasingly negative power shrinks toward 0 — exactly the “less weight further away” behaviour we want.
- $1 / (2\pi\sigma^2)$ — a normalizing constant that (in the continuous, ideal version) makes the entire surface integrate to 1, the same normalizing idea as dividing by 9 or 16 in earlier sections, just adapted for a smooth curve instead of a finite list of weights.

6.3 The slightly more careful version (the one to memorize)

The lecture's next slide writes essentially the same formula with the conventional $2\sigma^2$ in the denominator of the exponent (the standard form you'll see in every textbook and in OpenCV's documentation):

- Weight contribution of neighboring pixels by nearness.



$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Figure: The Gaussian filter. Source: S. Seitz

Navigation icons: back, forward, search, etc.

The standard 2D Gaussian function $G_{\sigma}(x,y)$, and an actual 5x5 kernel computed from it with $\sigma = 1$ (Source: S. Seitz).

$$G_{\sigma}(x, y) = 1 / (2\pi\sigma^2) \times e^{-(x^2+y^2)/(2\sigma^2)}$$

This is the formula you should keep as your reference — it's the standard “2D Gaussian / normal distribution” formula used everywhere in statistics and computer vision, just renamed x,y instead of u,v .

For a 5x5 kernel with $\sigma = 1$, plugging in every integer (x,y) pair from -2 to 2 and normalizing so the 25 values sum to 1 gives exactly the numeric grid shown in the figure above (center value 0.159, corners 0.003).

6.4 Python: building a Gaussian kernel from the formula

```
import numpy as np

def gaussian_kernel(size, sigma):
    """Builds a `size x size` Gaussian kernel directly from the formula:
     $G_{\sigma}(x,y) = 1/(2\pi\sigma^2) \times \exp(-(x^2+y^2)/(2\sigma^2))$ 
    """
    ax = np.arange(-(size // 2), size // 2 + 1) # [-2,-1,0,1,2] for size=5
    xx, yy = np.meshgrid(ax, ax) # (x,y) grid of offsets
    kernel = (1 / (2*np.pi*sigma**2)) * np.exp(-(xx**2+yy**2)/(2*sigma**2))
    return kernel / kernel.sum() # normalize so the weights sum to 1

g = gaussian_kernel(5, sigma=1)
print(np.round(g, 3))
print('Sum of weights:', g.sum())
```

Output:

```
[[0.003 0.013 0.022 0.013 0.003]
 [0.013 0.06 0.098 0.06 0.013]]
```



```
[0.022 0.098 0.162 0.098 0.022]
[0.013 0.06  0.098 0.06  0.013]
[0.003 0.013 0.022 0.013 0.003]]
Sum of weights: 1.0
```

This matches the lecture's 5×5 , $\sigma=1$ kernel exactly (the slide rounds the center to 0.159, which becomes 0.162 here only due to rounding — both are correct to the displayed precision).

Sanity check against OpenCV's built-in function

```
import cv2

cv_kernel_1d = cv2.getGaussianKernel(5, 1) # builds a 1D vector...
cv_kernel_2d = cv_kernel_1d @ cv_kernel_1d.T # ...combine into 2D
print(np.round(cv_kernel_2d, 3))
print('Matches our manual kernel:', np.allclose(g, cv_kernel_2d, atol=1e-3))
```

Output:

```
[[0.003 0.013 0.022 0.013 0.003]
 [0.013 0.06  0.098 0.06  0.013]
 [0.022 0.098 0.162 0.098 0.022]
 [0.013 0.06  0.098 0.06  0.013]
 [0.003 0.013 0.022 0.013 0.003]]
Matches our manual kernel: True
```

Why OpenCV builds a 1D kernel and multiplies it by its transpose

A 2D Gaussian is **separable**: blurring with one big 2D Gaussian kernel gives the exact same result as blurring once horizontally with a 1D Gaussian, then once vertically with the same 1D Gaussian. This is a major speed trick — filtering an $N \times N$ image with a $k \times k$ 2D kernel costs $O(N^2 k^2)$ operations, but doing it as two 1D passes only costs $O(N^2 k)$ — much faster for large kernels. The plain box filter is separable too, for the same reason (it's a constant in both directions).

6.5 Visualizing the Gaussian kernel as a surface

It helps enormously to literally see the bell shape the formula describes. Here's the same kernel from the code above, plotted as a 3D surface and as a grayscale image (brighter = larger weight):

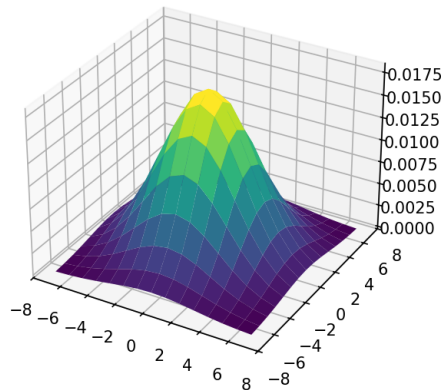
```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

kernel_big = gaussian_kernel(15, sigma=3) # bigger, easier to see the shape
ax_range = np.arange(-(15 // 2), 15 // 2 + 1)
xx, yy = np.meshgrid(ax_range, ax_range)

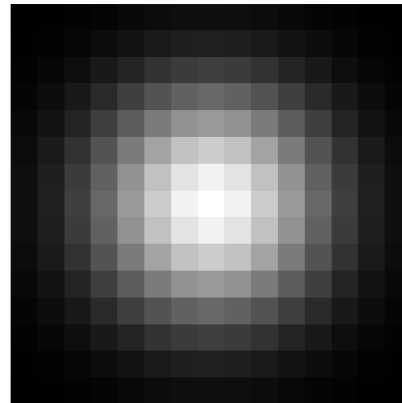
fig = plt.figure(figsize=(10, 4))
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
ax1.plot_surface(xx, yy, kernel_big, cmap='viridis')
ax1.set_title('Gaussian kernel - 3D view')

ax2 = fig.add_subplot(1, 2, 2)
ax2.imshow(kernel_big, cmap='gray')
```

```
ax2.set_title('Gaussian kernel - as an image')
plt.tight_layout()
plt.show()
```

Gaussian kernel $G_{\sigma}(x,y)$ - 3D view

Gaussian kernel - as an image (bright = high weight)



Output of the code above: the Gaussian formula visualized as a 3D bell surface (left) and as a grayscale image (right) — compare this to the lecture's diagram in Section 6.2.

6.6 Smoothing comparison: Gaussian vs box filter on a real photo

The lecture closes by comparing what each filter does to an actual textured photo (dry grass), using a small inset image to show the shape of the kernel that was used:

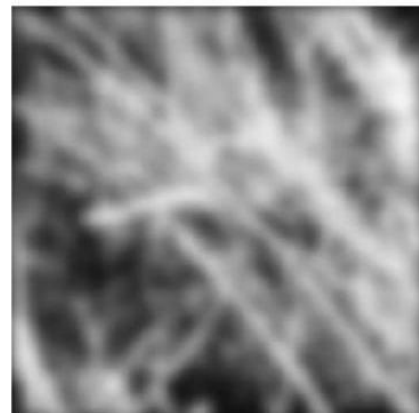


Figure: Smoothing with the Gaussian filter. Source: S. Seitz

Smoothing with the Gaussian filter: original (left) vs. Gaussian-blurred (right). Notice the blur looks smooth and natural (Source: S. Seitz).

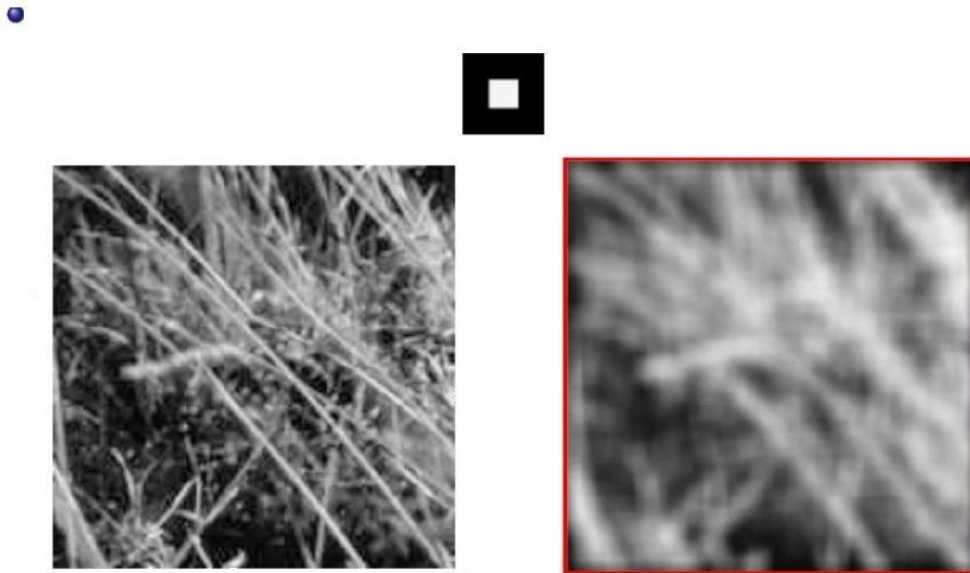


Figure: Smoothing with the Gaussian filter. Source: S. Seitz

Smoothing with the box filter (highlighted in red): compare the blur quality to the Gaussian result above — box-filter blur can look slightly more ‘smeared’ or blocky (Source: S. Seitz).

6.7 Python: reproducing this comparison

```
import cv2
import numpy as np

img = cv2.imread('grass.jpg', cv2.IMREAD_GRAYSCALE)

box_blurred = cv2.blur(img, (5, 5)) # box filter
gaussian_blurred = cv2.GaussianBlur(img, (5, 5), sigmaX=1) # Gaussian

cv2.imwrite('box_result.jpg', box_blurred)
cv2.imwrite('gaussian_result.jpg', gaussian_blurred)
```

To see this work on a controlled, synthetic test case (so we know exactly what the “clean” image looked like before adding noise), here's a complete denoising experiment, going all the way back to the noise formula from Section 1:

```
import numpy as np
import cv2

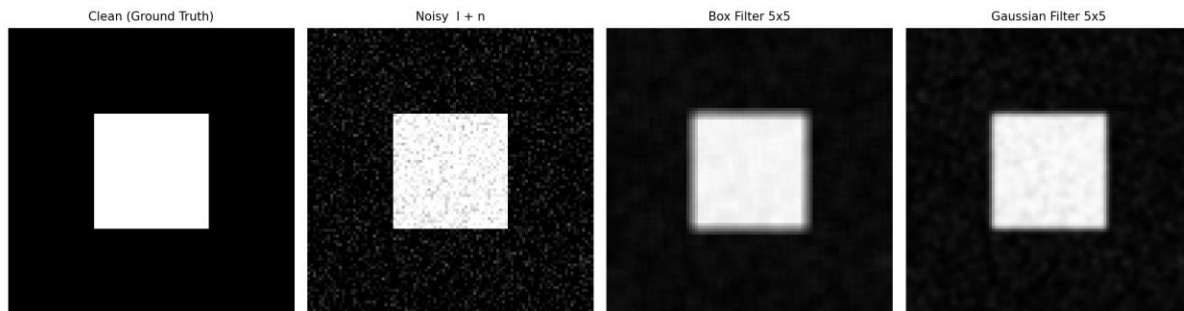
np.random.seed(42)

# 1) Clean image: a white square on a black background
clean = np.zeros((100, 100), dtype=np.float32)
clean[30:70, 30:70] = 255

# 2) Add noise --  $I_{\text{hat}}(x,y) = I(x,y) + n(x,y)$ , the formula from Section 1
noise = np.random.normal(0, 25, clean.shape).astype(np.float32)
```

```
noisy = np.clip(clean + noise, 0, 255).astype(np.uint8)

# 3) Denoise with each filter
box_filtered = cv2.blur(noisy, (5, 5))
gaussian_filtered = cv2.GaussianBlur(noisy, (5, 5), sigmaX=1)
```



Output of the code above: clean ground truth, the noisy version ($I + n$), and the result of denoising with each filter.

Both filters clearly reduce the speckled noise compared to the noisy input — this is the entire point of the lecture, now demonstrated end-to-end in code, starting from the noise model in Section 1 and ending with two different filtering strategies.

7. Summary: Box Filter vs. Gaussian Filter

Property	Box Filter	Gaussian Filter
Weighting	Equal weight everywhere in the window	Weight smoothly decreases with distance from center
Formula	$h(i,j)$ = average of all pixels in window	$G\sigma(x,y) = (1/2\pi\sigma^2) \cdot e^{-(x^2+y^2)/2\sigma^2}$
Parameter to tune	Window size (e.g. 3×3, 5×5)	σ (sigma) plus window size
Visual result	Can look blocky / smeared at high strength	Smooth, natural-looking blur
Separable?	Yes (constant in both directions)	Yes (product of two 1D Gaussians)
OpenCV function	<code>cv2.blur(img, (k,k))</code>	<code>cv2.GaussianBlur(img, (k,k), sigma)</code>
Best for	Quick, cheap smoothing; less concerned with quality	General-purpose denoising / pre-processing before edge detection

The one idea that ties the whole lecture together

Every technique you saw today — moving average, weighted average, correlation, box filter, Gaussian filter — is the *same algorithm*: slide a small weight matrix over the image, multiply-and-sum at every position. The only thing that changes from one filter to the next is **what weights go in**

the matrix. Once that clicks, the formulas stop being separate things to memorize and become one idea applied four different ways.

8. Putting It All Together: A Complete Mini-Lab

Here is a single, self-contained script that reproduces every concept from this lecture in order. Copy it into a file (e.g. `image_filtering_lab.py`) and run it — it needs only `numpy`, `opencv-python`, and `matplotlib` (install with `pip install numpy opencv-python matplotlib`).

```

"""
CS641 Lecture 03 - Image Filtering
A complete mini-lab covering every concept in the lecture, in order:
1. Simulate noise:          I_hat = I + n
2. 1D moving average
3. 1D weighted average
4. 2D correlation from scratch
5. Box filter (uniform weights)
6. Gaussian kernel + Gaussian filter
7. Side-by-side comparison on a synthetic noisy image
"""
import numpy as np
import cv2
import matplotlib.pyplot as plt

np.random.seed(0)

# -----
# 1. Image noise:  I_hat(x,y) = I(x,y) + n(x,y)
# -----
def add_gaussian_noise(image, sigma=25):
    noise = np.random.normal(0, sigma, image.shape).astype(np.float32)
    noisy = np.clip(image.astype(np.float32) + noise, 0, 255)
    return noisy.astype(np.uint8)

# -----
# 2 & 3. 1D moving average / weighted average
# -----
def weighted_average_1d(signal, weights):
    weights = np.array(weights, dtype=float)
    weights = weights / weights.sum()
    return np.convolve(signal, weights, mode='same')

# -----
# 4. 2D correlation from scratch:  h(i,j) = sum_k sum_l g(k,l) f(i+k,j+l)
# -----
def correlate2d(f, g):
    f = f.astype(float)
    gh, gw = g.shape
    pad_h, pad_w = gh // 2, gw // 2
    f_padded = np.pad(f, ((pad_h, pad_h), (pad_w, pad_w)), mode='constant')
    out = np.zeros_like(f)

```

```

    for i in range(f.shape[0]):
        for j in range(f.shape[1]):
            region = f_padded[i:i + gh, j:j + gw]
            out[i, j] = np.sum(g * region)
    return out

# -----
# 5. Box filter kernel
# -----
def box_kernel(size):
    return np.ones((size, size)) / (size * size)

# -----
# 6. Gaussian kernel:  $G_{\sigma}(x,y) = 1/(2\pi\sigma^2) * \exp(-(x^2+y^2)/(2\sigma^2))$ 
# -----
def gaussian_kernel(size, sigma):
    ax = np.arange(-(size // 2), size // 2 + 1)
    xx, yy = np.meshgrid(ax, ax)
    kernel = (1 / (2 * np.pi * sigma**2)) * np.exp(-(xx**2 + yy**2) / (2 *
sigma**2))
    return kernel / kernel.sum()

if __name__ == '__main__':
    # ---- 1D demo ----
    x = np.linspace(0, 1, 60)
    clean_1d = np.sin(2 * np.pi * 3 * x)
    noisy_1d = clean_1d + np.random.normal(0, 0.3, clean_1d.shape)
    smoothed_1d = weighted_average_1d(noisy_1d, [1, 4, 6, 4, 1])

    # ---- 2D demo: synthetic noisy image ----
    clean_2d = np.zeros((100, 100), dtype=np.uint8)
    clean_2d[30:70, 30:70] = 255
    noisy_2d = add_gaussian_noise(clean_2d, sigma=25)

    # Manual implementations (slow, but match the lecture formulas exactly)
    manual_box = correlate2d(noisy_2d, box_kernel(5))
    manual_gauss = correlate2d(noisy_2d, gaussian_kernel(5, sigma=1))

    # Fast OpenCV implementations (what you'd actually use in a project)
    cv_box = cv2.blur(noisy_2d, (5, 5))
    cv_gauss = cv2.GaussianBlur(noisy_2d, (5, 5), sigmaX=1)

    # ---- Show everything ----
    fig, axes = plt.subplots(2, 3, figsize=(13, 7))
    axes[0, 0].plot(x, clean_1d, label='clean')
    axes[0, 0].plot(x, noisy_1d, '.', label='noisy')
    axes[0, 0].legend()
    axes[0, 0].set_title('1D noise')
    axes[0, 1].plot(x, smoothed_1d)
    axes[0, 1].set_title('Weighted average [1,4,6,4,1]/16')
    axes[0, 2].axis('off')
    axes[1, 0].imshow(noisy_2d, cmap='gray')
    axes[1, 0].set_title('Noisy image')
    axes[1, 1].imshow(cv_box, cmap='gray')
    axes[1, 1].set_title('Box filter')
    axes[1, 2].imshow(cv_gauss, cmap='gray')

```

```
axes[1, 2].set_title('Gaussian filter')
for ax in axes[1, :]:
    ax.axis('off')
plt.tight_layout()
plt.show()

interior_box = manual_box[5:-5, 5:-5]
interior_cv = cv_box[5:-5, 5:-5].astype(float)
print('Box filter match:', np.allclose(interior_box, interior_cv, atol=2))
```

Output:

```
Box filter match: True
(plus a matplotlib window showing all six plots described above)
```

What's Next: Edge Detection

The lecture ends by pointing to next class's topic: **Edge Detection**. There's a nice continuity here worth previewing: averaging filters (this lecture) are *low-pass* filters — they keep slow, smooth changes and remove fast, jittery ones (noise). Edge detection works in the opposite direction: edges *are* the fast, jittery changes in brightness (a sudden jump from dark to light), so edge-detection filters are *high-pass* filters — they ignore the slow-changing smooth regions and highlight exactly the sudden jumps that this lecture's filters were designed to smooth away. The kernel-sliding mechanics you just learned (multiply-and-sum over a small window) carry over directly; only the numbers inside the kernel change.

End of notes for Lecture 03 — Image Filtering.